

CLAIMS CONCIERGE

Architecture & Development Standards

MVP Reference Document — v0.1

Date: **May 13, 2026** Prepared by: **Sayone Technologies** Client: **Claims Concierge**

1. Purpose

This document captures the architecture, technology choices, and engineering standards for the Claims Concierge MVP. It serves as the single shared reference for the build team during the initial two-week development sprint and the subsequent productionization phase.

The DV (Diminished Value) and Total Loss report generation logic is intentionally summarized at a high level here. The detailed specification — inputs, model prompts, structured outputs, validation rules, and report templates — will arrive in a separate requirements document and is out of scope for this file.

2. Architecture Overview

The platform follows a clean, single-deployment architecture with one application service that serves both API and WebSocket traffic, a worker process for the asynchronous AI pipeline, and managed GCP services for data and storage. There are two trigger paths into the system: an automated path from HubSpot (when an appraiser uploads a repair bill on a custom object), and a manual path from the dashboard (when a Super Admin or Standard User creates a new report). Both paths converge on the same Celery-driven pipeline.

2.1 Component Roles

- **Caddy** — reverse proxy with automatic HTTPS, terminates TLS for both HTTP and WebSocket traffic.
- **FastAPI application** — handles REST APIs, WebSocket endpoints, HubSpot webhook ingestion, authentication, and user management. One codebase, one deployable.
- **Celery worker** — executes the multi-step DV and Total Loss generation pipeline. Same codebase as FastAPI, different endpoint.
- **Redis** — Celery broker, pub/sub channel for progress events, and cache for sessions and idempotency keys.
- **PostgreSQL (Cloud SQL)** — relational store for users, reports, pipeline step state, and audit log.
- **Cloud Storage (GCS)** — persistent file storage for generated PDFs and source repair bills, with strict IAM and signed URLs for downloads.
- **LiteLLM** — provider-agnostic LLM gateway, allowing model selection to flex between Anthropic, OpenAI, and Gemini without pipeline code changes.
- **LangSmith** — LLM observability: tracing, prompt versioning, latency, and cost per call, evaluation feedback.
- **Secret Manager** — secure storage for API keys, OAuth tokens, and database credentials.

2.2 Request Flow Summary

HubSpot-triggered: A repair bill is uploaded to a HubSpot custom object. HubSpot fires a signed webhook to Caddy → FastAPI. FastAPI verifies the HMAC signature, persists a new report record, enqueues a Celery job, and responds immediately. The worker runs the multi-step LLM pipeline, publishing progress events to Redis pub/sub. On completion, the worker uploads the PDF to GCS, writes structured data and audit entries to Postgres, and FastAPI calls back to HubSpot via OAuth to attach the file and update the custom object.

Dashboard-triggered: A logged-in user creates a new report from the dashboard, uploading the repair bill via the UI. The same downstream pipeline runs. Progress is streamed live to the user's browser over WebSocket via the Redis pub/sub bridge.

3. Technology Stack

The stack is selected for AI-product velocity, GCP-native operation, and team familiarity. Choices marked TBD will be confirmed during the first week of the build.

Layer	Selection	Notes
Frontend	Next.js 15 + TypeScript	App Router, Tailwind, shadcn/ui
Backend	FastAPI + Pydantic v2	Python 3.12+, async-first
ORM	SQLAlchemy 2.0 (async)	Alembic migrations
Database	PostgreSQL on Cloud SQL	JSON columns + full-text search
Pipeline / Queue	Celery + Redis broker	Step state in Postgres
Real-time	WebSocket + Redis pub/sub	Live progress to UI
Cache / Sessions	Redis (Memorystore)	Separate instance in prod
File Storage	Google Cloud Storage	Signed URLs, strict IAM
PII Detection & Redaction	Microsoft Presidio (presidio-analyzer + presidio-anonymizer)	Open source, Python-native; reversible tokenize-call-detokenize pattern
LLM Abstraction	LiteLLM	Claude, OpenAI, Gemini swappable
LLM Observability	LangSmith Cloud	Free Developer tier: traces, prompts, cost
Auth	FastAPI Users (Python, OSS)	Users in their own Postgres; JWT issued by the backend
Reverse Proxy	Caddy	Auto-HTTPS
Containers	Docker / Docker Compose	Same image for app + worker
IaC	Terraform	Post-MVP phase
CI/CD	GitHub Actions	Pre-commit + branch protection

Layer	Selection	Notes
Error Tracking	Sentry	Frontend + backend
Email (Prod)	SendGrid	Transactional;
Email (Dev)	MailHog	Catches SMTP locally; web UI to inspect

4. Application Structure

The application is structured as a single deployable service with two process entrypoints — a web process (Uvicorn) and one or more Celery workers. The codebase is organized by domain with clear internal boundaries so that future extraction into separate services, if ever needed, is a refactor rather than a rewrite.

4.1 Module Layout

- `app/api/` — FastAPI routers (HTTP and WebSocket endpoints)
- `app/domain/` — pure business logic, framework-independent (claims, reports, users, audit)
- `app/integrations/hubspot/` — HubSpot webhook handlers and OAuth callback client
- `app/integrations/llm/` — LiteLLM wrapper and prompt management
- `app/workers/` — Celery task definitions and pipeline orchestration
- `app/db/` — SQLAlchemy models and Alembic migrations
- `app/ws/` — WebSocket handlers and Redis pub/sub bridge
- `app/core/` — configuration, logging, authentication, error handling

4.2 Deployment Model

For the initial two-week sprint, the application runs as a Docker Compose stack mirroring the production topology — Caddy, FastAPI (Uvicorn), Celery worker(s), and containerized Postgres and Redis for development. After MVP code completion, the deployment is migrated to a Compute Engine VM running the same Docker Compose stack and managed Cloud SQL, Memorystore, and GCS. A second phase migrates the workloads to GKE Autopilot.

5. HubSpot Integration

5.1 Trigger

The HubSpot custom object that represents a claim is monitored for a property change indicating a repair bill upload. HubSpot fires a workflow webhook to a dedicated FastAPI endpoint, including the custom-object record ID, appraiser name, and the basic claim fields needed to start the pipeline.

5.2 Authentication

- Inbound (HubSpot → us): HMAC-SHA256 signature verification on every webhook using HubSpot's native signed-webhook pattern.
- Outbound (us → HubSpot): OAuth access token issued by a HubSpot Private App, scoped to the custom object, Files API, and timeline endpoints. Tokens refresh automatically.

5.3 Async Contract

Pipeline runs are long-running. FastAPI acknowledges the webhook immediately with a 202 response and proceeds asynchronously. On completion, the worker uploads the PDF to HubSpot Files, updates the custom object properties with structured data, and writes a timeline entry. On failure, an error status and reason are written back so the appraiser sees what went wrong.

5.4 Idempotency and Retries

Duplicate webhook deliveries (e.g., HubSpot retries) are deduplicated using the custom-object record ID combined with the hash of the uploaded repair bill. Pipeline tasks are written idempotently and retried with exponential backoff using Celery's built-in retry mechanism.

6. Async Pipeline & Real-Time Status

6.1 Pipeline Shape

DV and Total Loss reports are generated by a multi-step Celery chain. Each step is an atomic, idempotent task. Step boundaries persist state to Postgres so that the pipeline is resumable and auditable. Step-by-step prompts and integration details will be specified in the dedicated DV/Total Loss requirements document.

6.2 State Model

- reports — high-level record per generation (status, source, requested-by, timestamps)
- report_steps — one row per pipeline step (name, status, started_at, completed_at, output reference)
- audit_log — append-only record of every report event (create, step transition, download, callback)

6.3 Real-Time Progress to UI

Celery workers publish progress events to a Redis pub/sub channel keyed by report ID. A WebSocket handler in FastAPI subscribes to the channel and forwards events to the user's browser session. Events include current step name, progress percent (when measurable), and final status.

6.4 In-Progress Visibility Rules

Reports appear in the dashboard list as soon as they are created, with a clear status badge — Queued, Generating, Completed, or Failed. Rows in non-terminal states are locked: no edits, downloads, or deletions are permitted until the report transitions to Completed. This guards against partially-generated outputs being treated as final while still giving users full visibility into queue status.

7. Authentication & Authorization

7.1 Roles (MVP)

Role	Permissions
Super Admin	Full access — create users, manage all reports, view audit log, regenerate, and delete
Standard User	Create / view / download / delete / regenerate own reports

7.2 Authentication Stack

- Authentication is implemented with FastAPI Users — an open-source Python library that integrates natively with FastAPI.
- User records, hashed passwords (argon2), and roles live in the same PostgreSQL database as the rest of the application; no third-party auth vendor or per-seat cost.
- Login is email and password. The backend issues a signed JWT; the Next.js frontend attaches it to subsequent API calls.
- No MFA in MVP. Strongly recommended for the next phase, especially for Super Admin.
- No custom session timeout configuration in MVP — sensible default JWT lifetime applies.

7.3 User Onboarding Flow

Users are onboarded by invitation. There is no public self-serve signup.

- Super Admin creates a user record by entering their name, email, and role in the dashboard.
- The system generates a single-use, time-limited registration token bound to that user and emails the user a registration link.
- The user opens the link, lands on a registration page, sets their own password (with strength rules), accepts terms if applicable, and the account is activated.
- Registration links expire after a configurable window (default 7 days). Super Admin can resend the invite to issue a fresh token.
- Email addresses are verified implicitly through the registration flow — only the holder of the invited mailbox can complete signup.

7.4 Password Reset Flow

Self-service password reset is supported via email.

- The login page exposes a Forgot Password link. The user enters their email.
- If the email matches an active user, the system emails a single-use, time-limited reset link. The same response is shown either way to avoid leaking which addresses exist.
- The user opens the link, sets a new password, and is signed in.
- Reset links expire after a short window (default 60 minutes) and are invalidated on use.
- Super Admin can also force a password reset for any user, which sends them a fresh reset email.

7.5 Email Delivery

- All transactional email (invitations, password resets, status notifications) is sent via SMTP.
- Production uses SendGrid (SMTP relay or REST API).
- Local development uses MailHog — a containerized SMTP catcher with a web UI at <http://localhost:8025>. No emails leave the developer's machine in dev or test.
- Email templates are version-controlled in the repo (Jinja2 templates), with both HTML and plain-text variants.

7.6 Audit Trail

Every authentication event, user creation, report generation, status transition, and download is recorded in the `audit_log` table with timestamp, user ID, and action context. The audit log is queryable from the Super Admin view.

8. UI Components & Patterns

The dashboard is built with Next.js, Tailwind CSS, and shadcn/ui. Components follow a consistent design system with reusable building blocks. The list below captures the components and patterns required for the MVP.

8.1 Layout & Navigation

- Application shell with sidebar navigation and top bar
- User menu with profile and logout actions
- Breadcrumb navigation on detail pages
- Responsive collapse for narrow viewports

8.2 Authentication Screens

- Login screen — email and password
- Forgot Password screen — request reset link by email
- Reset Password screen — set new password from emailed link
- Invite Registration screen — set initial password and complete onboarding from emailed link
- Expired-link / invalid-token states with clear next-action guidance

8.3 Reports List View

- A paginated table of all reports the user can access
- Columns: Report ID / Claim reference, Type (DV / Total Loss), Source (HubSpot / Manual), Status badge, Created date, Created by, Actions
- Filters: status, type, source, date range, created-by
- Search by claim reference/report ID
- Sort by date, status, and type
- Status badge colors: neutral for Queued, accent for Generating, green for Completed, red for Failed

8.4 Report Detail View

- Header with report ID, type, status badge, source indicator
- Structured data display for the generated report fields
- Download button (PDF) for completed reports
- Regenerate action for completed or failed reports
- Audit timeline showing each step transition with timestamps
- Source repair bill download link (signed URL)

8.5 New Report Form

- Form to manually trigger generation from the dashboard
- Report type selector (DV / Total Loss)
- Required fields: claim reference, appraiser name, basic claim info

- Repair bill file upload
- Submit creates a report in Queued status and routes the user to the Report Detail view to watch progress

8.6 Async Task Progress Tracker

This is a core UI primitive used in both the Report Detail view and inline in the Reports List. It surfaces live status from the asynchronous pipeline, irrespective of trigger source.

- Live status indicator: Queued, Generating, Completed, Failed
- Step progress visualization: a horizontal step list showing the current and completed steps of the pipeline
- Source indicator: badge or icon showing whether the run was triggered from HubSpot or from the dashboard
- Live updates over WebSocket — UI re-renders as worker events arrive on the Redis pub/sub channel
- Elapsed time counter while the pipeline is running
- Failure reason and retry call-to-action when status is Failed
- Read-only / locked treatment when the report is in a non-terminal state: no edit, download, or delete affordances visible
- Graceful degradation: if the WebSocket disconnects, the component falls back to polling at a low frequency and re-establishes the socket when possible

8.7 User Management (Super Admin)

- User list with role, status, and last login
- Create user dialog (email, role)
- Reset password action
- Deactivate/reactivate user

8.8 Audit Log Viewer

- Filterable, paginated view of audit events
- Filters: user, action type, report, date range
- Export to CSV (post-MVP)

8.9 In-App Notifications

- Toast notifications for state transitions (report ready, generation failed, user created)
- Notification center icon in the top bar with unread count

8.10 Universal States

- Empty states with clear next-action guidance
- Skeleton loaders for list and detail views
- Inline error messages with retry where applicable
- Form validation with field-level errors and a top-of-form summary

9. Design System & Branding

The dashboard's visual identity is aligned with the existing Claims Concierge brand at <https://www.claimsconcierge.io/>, so the product feels like a natural extension of the marketing

site. The design system is built as part of Phase 1 and is the foundation for every UI component shipped afterward.

9.1 Brand Reference

- Primary brand reference: <https://www.claimsconcierge.io/>
- Logo asset is sourced directly from the live site and exported as SVG for crisp rendering at any size
- Exact color hex values, font families, and spacing tokens are sampled from the live site during the design-system setup task and committed to a single source of truth in the repo

9.2 Color Tokens

Initial token shape (final hex values to be confirmed by sampling the live site).

Token	Role	Indicative Source
brand.primary	Main brand color (headers, key actions, links)	Claims Concierge navy / dark blue from header & footer
brand.accent	CTAs, highlights, primary buttons	Claims Concierge "Get Started" CTA color
surface.bg	Page background	White
surface.card	Cards, table rows, modals	Off-white / very light neutral
text.primary	Body and heading text	Near-black neutral
text.secondary	Subtitles, captions, meta	Mid-gray neutral
border.subtle	Dividers, table borders	Light gray
status.queued	Async task badge — Queued	Neutral gray
status.progress	Async task badge — Generating	Brand accent
status.success	Async task badge — Completed	Green
status.error	Async task badge — Failed	Red
status.warning	Warnings, soft alerts	Amber

9.3 Typography

- Font family: matched to the marketing site (modern sans-serif). Final family confirmed by inspecting the live site CSS.
- Type scale: H1 32, H2 24, H3 20, body 16, caption 14, micro 12 (all rem-based, responsive).
- Line heights: 1.2 for headings, 1.5 for body.
- Font loading via next/font for self-hosted, fast first paint.

9.4 Layout & Spacing

- 4-point base unit; spacing scale 4 / 8 / 12 / 16 / 24 / 32 / 48 / 64 px.
- Container max-width: 1280px for dashboard content, 1440px for full-width data views.

- Border-radius scale: 4 (inputs), 8 (cards), 12 (modals).
- Elevation: minimal shadows; rely on borders and surface contrast for separation.

9.5 Component Library

- shadcn/ui as the base component set, themed to brand tokens.
- Components live in app/components/ui (primitives) and app/components/features (composed).
- Every primitive is rebrandable through the token layer — no hardcoded colors anywhere.
- Storybook (optional, post-MVP) for visual regression and component documentation.

9.6 Accessibility

- WCAG 2.1 AA targeted minimum.
- Color contrast ratios validated for brand tokens before lock-in.
- Keyboard navigation, focus rings, and ARIA labels for interactive components by default.

10. Data & Storage

10.1 PostgreSQL

- Tables: users, roles, reports, report_steps, audit_log, hubspot_webhooks (raw event log for idempotency)
- Alembic migrations from the first table onwards — no ad-hoc schema changes
- Indexes on report status, source, created-by, and HubSpot record ID for the list view query patterns

10.2 Cloud Storage

- Bucket per environment with strict IAM
- Object paths organized as: /reports/{report_id}/source/repair_bill.pdf and /reports/{report_id}/output/{type}.pdf
- Signed URLs (short expiry) for downloads — no public buckets
- Server-side encryption at rest (managed by GCS)

10.3 Retention

Standard retention policy applies to all data. Audit log records are retained indefinitely. Source files and generated PDFs follow the agreed claim-handling retention window (to be confirmed with Claims Concierge). Soft-deletion is preferred over hard-deletion for any record referenced by an audit entry.

11. AI / LLM Layer

11.1 Provider Abstraction

All LLM calls go through LiteLLM, a thin abstraction that exposes a single client interface across providers (Anthropic, OpenAI, Gemini, and others). The provider, model, and temperature for each step are configured at the prompt level. This satisfies the requirement to keep model selection open and to apply for startup credits across providers.

11.2 Observability

LangSmith Cloud is used as the LLM observability platform. Every LLM call is traced through the LiteLLM integration, capturing model, prompt version, input, output, token usage, latency, and cost. Traces are tagged with feature, claim reference, environment, and tenant for cost attribution. Prompt versioning is managed in LangSmith Prompts so prompt iteration is reviewable and reversible. The free Developer tier covers early-stage usage; paid tiers scale cleanly with no self-hosting overhead.

11.3 Cost Visibility (Phase 1)

In line with the agreed cost-monitoring approach, the MVP does not include a custom in-app cost console. Visibility is provided through LangSmith Cloud, the Anthropic (or the model we use) consoles, and GCP Billing. A weekly cost report — total spend, breakdown by feature and model, cost per report, forecast vs. budget — is generated from these sources and shared with the Claims Concierge team.

11.4 Structured Outputs

Pipeline steps that produce structured data use JSON schemas enforced via Instructor (or the provider's native JSON mode). Output validation is treated as part of the step contract; a step is not considered complete until its output validates.

12. Development Standards

12.1 Code Quality

- Python: Ruff for linting and formatting, mypy for type checking
- TypeScript: ESLint and Prettier, with strict mode enabled
- All code is type-annotated; new code without types fails CI

12.2 Pre-Commit Hooks

Configured via the pre-commit framework and installed on day one. Hooks run automatically on every commit.

- Ruff (lint and format) for Python
- mypy for Python type checking
- ESLint and Prettier for TypeScript / React
- gitLeaks or detect-secrets to prevent committing credentials
- commitlint to enforce Conventional Commits
- File hygiene: trailing whitespace, end-of-file fixer, YAML / JSON validation

12.3 Testing

- Backend: pytest with pytest-asyncio for async tests
- Frontend: Vitest for unit and component tests
- **End-to-end: Playwright covering the critical user flows (login, create report, view progress, download)**
- Integration tests run against ephemeral Postgres and Redis containers
- Coverage gate starts at 70% and rises as the codebase matures

12.4 Git Workflow

- Trunk-based development on main with feature branches

- Conventional Commits for all commits
- Pull requests required for any change to main; direct push disabled
- CI must pass before merge; at least one reviewer's approval is required

12.5 Logging

- JSON-structured logs across backend services with correlation IDs
- Correlation IDs propagated through HTTP, webhook, and Celery boundaries
- Sensitive fields (PII, repair bill content) redacted before logging

12.6 Configuration & Secrets

- All configuration via environment variables, loaded through Pydantic Settings
- .env.example checked in; .env never committed
- Production secrets stored in GCP Secret Manager; the app reads them at startup

13. CI/CD & Infrastructure

13.1 GitHub Actions Pipeline

The pipeline runs on every pull request and on merges to main.

- Static analysis: Ruff, ESLint, mypy, TypeScript compilation
- Unit tests: pytest and Vitest
- Integration tests: pytest against a test Postgres and Redis
- End-to-end tests: Playwright on PR and nightly
- Container security scan: Trivy on built images
- Dependency scan: GitHub Dependabot and/or Snyk
- Build and push Docker images to GCP Artifact Registry on merge
- Deploy to staging on merge to main; production on tagged release

13.2 Infrastructure Phasing

Phase 1 — MVP build (weeks 1–2): Application code is developed against a local Docker Compose stack mirroring the production topology. No production infra is provisioned yet.

Phase 2 — Production cutover (week 2+): Terraform provisions GCP resources: a Compute Engine VM running Docker Compose (Caddy + FastAPI + Celery), Cloud SQL for Postgres, Memorystore for Redis, GCS for files, Secret Manager for credentials, and IAM scoped to least privilege.

14. MVP Scope & Phasing

The work is divided into phases. Phase 1 deliberately does not build the DV or Total Loss generation pipeline — that work waits on the detailed report specification, which is being prepared separately by Claims Concierge. Phase 1 focuses on every other layer that the report generation will eventually plug into.

14.1 Phase 1 — In Scope

Design system and branding: color tokens, typography, spacing, component library, all sampled and aligned with <https://www.claimsconcierge.io/>

- Authentication with FastAPI Users — Super Admin and Standard User roles, JWT-based session
- Invite-based onboarding: Super Admin creates user, system emails registration link, user clicks, and sets own password
- Self-service password reset via email (single-use, time-limited links)
- Email infrastructure: SendGrid in production, MailHog in local dev
- Versioned email templates (HTML + plain text) for invite, password reset, and status notifications
- Dashboard shell: navigation, layout, user menu, responsive frame
- Reports list view (paginated, filterable, sortable, status badges) — wired to backend, populated by stub pipeline runs
- Report detail view, including the Async Task Progress Tracker UI driven by WebSocket events
- New Report form (manual creation flow)
- HubSpot integration scaffolding: signed webhook receiver that creates a report record, OAuth callback skeleton — pipeline body is a stub that simulates step progress and emits events end-to-end
- WebSocket progress channel via Redis pub/sub, wired end-to-end with the stub pipeline, so the UI is fully exercised
- User management screens (Super Admin)
- Audit log capture across all events and Super Admin viewer
- Pre-commit hooks and CI pipeline from day one
- Local Docker Compose dev environment mirroring production topology

14.2 Phase 2 — DV and Total Loss Generation

- Implement the actual multi-step LLM pipeline for DV reports
- Implement the actual multi-step LLM pipeline for Total Loss reports
- Prompt engineering, structured output schemas, validation rules
- Final PDF rendering matched to the agreed report template
- HubSpot callback wiring (upload PDF, write structured data, timeline entry)
- LangSmith Cloud instrumentation, traces, prompt versioning
- End-to-end acceptance testing against sample claims

14.3 Phase 3 — Production Infrastructure

- Terraform-managed GCP environment
- Caddy + Docker Compose on Compute Engine for first production cutover
- Cloud SQL, Memorystore, GCS, and Secret Manager are all provisioned via IaC
- Staging environment with automated deploys from main
- Production deploys via tagged release

14.4 Out of Scope for MVP

- MFA / SSO
- Public self-serve signup (users join only via Super Admin invite)
- Custom in-app cost console (covered by LangSmith and vendor dashboards for now)
- Internationalization and localization
- Mobile-optimized UI beyond responsive layout

15. Data Protection & PII

The platform handles personally identifiable information (PII) at every stage of the DV and ACV report lifecycle. The following section captures the classification, controls, and obligations that apply across the system.

PII Inventory

The following data is classified as PII and handled with the controls described below:

- Vehicle owner name, address, phone number, and email
- Vehicle Identification Number (VIN)
- License plate number
- Insurance policy number and claim number
- Appraiser identity and contact information
- Repair bill contents (often containing all of the above, plus financial detail)
- Photos and supporting documents that may incidentally include identifying information
- Financial details: pre-accident value, repair cost, settlement amount

PII in the Pipeline

PII appears at multiple points:

- Inbound HubSpot webhook payload
- Source repair bill stored in GCS
- Inputs and outputs of LLM pipeline steps
- Structured fields persisted to Postgres
- Generated PDF report stored in GCS
- Outbound HubSpot callback (PDF upload and custom object update)

Each of these surfaces is subject to the controls below.

Encryption

All data is encrypted at rest using the native encryption of Cloud SQL and Cloud Storage. All network traffic is over TLS — between client and Caddy, between Caddy and the application, between the application and LLM providers, and between the application and HubSpot. For especially sensitive identifiers (VIN, claim number), column-level encryption in Postgres may be added in Phase 2 if the risk review calls for it.

Access Control

- IAM on GCS buckets follows least privilege. Application service accounts have read/write access only to the paths they need.
- No GCS bucket is public. All downloads are served via signed URLs with short expiry (default 5 minutes).
- Database access is restricted by service account; no developer accesses production data directly.
- Every report view, download, and callback is recorded in the audit log with timestamp, user, and action.

Retention and Deletion

- Source repair bills and generated PDFs are retained for the agreed claim-handling window (to be confirmed with Claims Concierge).
- Audit log records are retained indefinitely (to be confirmed with Claims Concierge).
- A documented data-subject deletion process supports requests under CCPA and other applicable laws. Deletion removes PII fields and source files while preserving audit references in a redacted form.

Logging and Tracing Hygiene

- PII fields are never written to application logs in plain form. A central log filter masks known PII fields before they are emitted.
- LangSmith traces redact PII before transmission. Trace inputs and outputs go through the same masking layer as application logs.
- Correlation IDs and report identifiers are safe to log; raw PII is not.

Backups

- Cloud SQL automated backups are encrypted, access-controlled, and retained per the agreed policy.
- GCS object versioning is enabled with a bounded retention window for the report buckets.

16. Coming in Separate Documents

The following items are intentionally summarized in this document and will be specified in detail elsewhere.

- DV report requirements: inputs, fields, pipeline steps, prompts, output format, validation rules, sample reports
- Total Loss report requirements: same level of detail as DV
- Repair bill format and parsing approach (PDF, scanned image, OCR strategy)
- LLM prompt library and prompt versioning conventions
- UX wireframes and visual design specification
- Detailed test plan including acceptance criteria for each report type

